SDLC

ciclo de vida del Desarrollo de software

¿Qué es el SDLC?

Es un marco estructurado y sistemático que organiza el proceso de desarrollo de software en fases claras y manejables.

Objetivo Principal

Que planifica, crea, prueba y despliega software de alta calidad de manera eficiente, reduciendo riesgos y costos, cumpliendo los requisitos del cliente.

Beneficios Clave

- Reduce riesgos y costos
- Mejora la calidad del producto
- Facilita la comunicación del equipo
- Permite entregas predecibles
- Asegura satisfacción del cliente

CICLO DE VIDA DEL DESARROLLO DE SOFTWARE

Las 7 etapas principales del SDLC

1. Planificación
Define el alcance, objetivos y viabilidad del proyecto

2. Requisitos
Recopila los requisitos funcionales y no funcionales

3. Diseño
Arquitectura HLD/LLD y patrones como MVC

4. Implementación
Es código limpio y buenas prácticas

5. Pruebas
Tests unitarios, integración y calidad

6. Despliegue
CI/CD y contenedores (Docker)

7. Mantenimiento
Correctivo y evolutivo

ALCANCE DEL PROYECTO

¿Por qué es importante definirlo bien?

Evita cambios no planificados, controla costos, alinea expectativas y permite planificar recursos correctamente.

4. Indica si cada requisito es funcional o no funcional:
a) El sistema debe permitir iniciar sesión con correo y contraseña. (FUNCIONAL)
b) El sistema debe responder en menos de 2 segundos.(NO FUNCIONAL)
c) El sistema debe permitir registrar pagos mensuales. (FUNCIONAL)
d) El sistema debe soportar 500 usuarios conectados al mismo tiempo.
(NO FUNCIONAL)

HLD (Alto Nivel): Es como una visión general del sistema qué módulos existen, cómo se comunican entre sí, qué tecnologías se usan. Es el "mapa grande".

LLD (Bajo Nivel): detalle técnico de cada módulo cómo funciona por dentro, clases, métodos, estructuras de datos. Es el "plano detallado".

¿Cuál patrón arquitectónico separa la aplicación en Modelo, Vista y Controlador?
RTA/
MVC (Model-View-Controller).

¿Qué es Clean Code?
RTA/
Código limpio: Es fácil de leer, entender y mantener. Usa nombres claros para variables y funciones, evita código repetido y cada función hace una sola cosa.

¿Qué pruebas validan funciones o métodos individuales?
RTA/
Pruebas unitarias (Unit Tests). Verifica que cada función o método funcione correctamente de forma aislada.

¿Qué significa CI/CD?
RTA/

- CI : Integración Continua (Continuous Integration): integra y prueba el código automáticamente cada vez que hay cambios.
- CD : Entrega/Despliegue Continuo (Continuous Delivery/Deployment): automatiza el proceso de llevar el software a producción.

¿Qué herramienta empaqueta aplicaciones en contenedores?
RTA/
Docker.

Mantenimiento correctivo vs evolutivo:
RTA/

- Correctivo: arregla errores o bugs que ya existen en el sistema.
- Evolutivo: agrega nuevas funciones o mejoras al sistema según nuevas necesidades.

Imagina que una aplicación bancaria falla porque no soporta muchos usuarios conectados al tiempo.

¿Qué tipo de requisito falló?
RTA/
Un requisito no funcional específicamente de escalabilidad y rendimiento.

¿En qué fase debió detectarse?
RTA/
En la fase de Análisis de Requisitos, donde se deben definir los límites de carga del sistema. También se pudo confirmar en la fase de Pruebas mediante pruebas de carga (load testing).